Agentic架构 8: Episodic + Semantic Memory Stack

> **持久化记忆**。标准聊天机器人的记忆是短暂的，仅持续单个会话。为了构建能够与用户共同学习和成长的个性化智能体，需要一个更强大的解决方案。 实现一种结构化的记忆架构，该架构模仿人类认知，融合两种不同类型的记忆：
>
> 1. **情景记忆：** 这是**对特定事件或过往交互的记忆**。它回答的是 **“发生了什么？”** 这个问题。为此，使用**向量数据库**来检索与当前话题相关的历史对话。
> 2. **语义记忆：** 这是**从这些事件中提取的结构化事实、概念和关系的记忆**。它回答的是 **“我知道什么？”** 这个问题。为此，使用**图数据库（Neo4j）**，因为它在管理和查询复杂关系方面表现出色。
>
> 通过结合这两种记忆，智能体不仅能**回顾过往对话**，还能**构建**关于用户及世界的、丰富互联的**知识库**，从而实现深度个性化和具有上下文感知能力的交互。

定义

情景记忆 + 语义记忆堆栈（Episodic + Semantic Memory Stack ） 是一种维护两种长期记忆的智能体架构。**情景记忆**存储按**时间顺序**排列的经验记录（例如对话历史摘要），通常基于**语义相似性**进行检索。**语义记忆**则以**知识库**（通常是图数据库）的形式存储提取出的**结构化知识**（事实、实体、关系）。

高级工作流程

1. **交互：** 智能体与用户进行**对话**。
2. **记忆检索（回忆）：** 对于新的用户查询，智能体首先**查询两个记忆系统**。
   - 它检索**情景向量存储**中与当前**话题相似**的历史对话。
   - 它查询**语义图数据库**，获取**与查询相关**的实体和事实。
3. **增强生成：** 检索到的记忆被添加到**提示词的上下文**中，使大语言模型能够生成一个知晓过往交互和已学习事实的响应。
4. **记忆创建（编码）：** 交互完成后，后台进程分析此次对话。
   - 它会为本次交互创建一个简洁的**摘要**（作为新的**情景记忆**）。
   - 它会**提取关键的实体和关系**（作为新的**语义记忆**）。
5. **记忆存储：** 新的**情景摘要**被嵌入并保存到**向量存储**中。新的**语义事实**作为节点和边被写入**图数据库**。

适用场景 / 应用

- **长期个人助理：** 能够在数周或数月内记住你的偏好、项目和个人细节的助理。
- **个性化系统：** 能够记住你风格的电商机器人，或能记住你学习进度和薄弱环节的教育辅导机器人。
- **复杂研究智能体：** 在探索文档时构建主题知识图谱，从而能够回答复杂的多跳问题的智能体。

优势与劣势

- **优势：**
  - **真正的个性化：** 实现了远超单次会话上下文窗口限制的、**可持久存在的上下文和学习**能力。
  - **丰富的理解力：** **图数据库**使智能体能够**理解和推理**实体之间的复杂关系。
- **劣势：**
  - **复杂性：** 与简单的无状态智能体相比，这是一种构建和维护起来都**复杂**得多的架构。

- **记忆膨胀与修剪：** 随着时间的推移，记忆存储可能会变得非常庞大。**制定总结、整合或修剪旧有/无关记忆**的策略，对于长期性能至关重要。

阶段 0：基础与环境搭建

安装所有必要的库，包括**向量数据库和图数据库**的驱动程序，并配置好 API 密钥。

```
pip install -q -U langchain-nebius langchain langgraph rich python-dotenv
langchain_community langchain-openai neo4j faiss-cpu tiktoken
```

```
"""
记忆堆栈架构 (Memory Stack Architecture) - 具有多层次记忆系统的智能体
记忆堆栈架构是一种高级智能体设计模式，集成了多种记忆类型：
- 短期记忆：对话历史和当前会话上下文
- 长期记忆：向量数据库中存储的知识和事实
- 工作记忆：当前任务的状态和执行轨迹
- 知识图谱：实体关系网络，支持推理和联想
"""

import os
import uuid
from typing import List, Dict, Any, Optional, Tuple
from dotenv import load_dotenv

# ==================== Pydantic 组件导入 ====================
# Pydantic用于数据验证、配置管理和结构化输出定义
from pydantic import BaseModel, Field

# ==================== LangChain 组件导入 ====================
# LangChain提供LLM抽象、向量存储、图数据库集成等核心功能
from langchain_nebius import ChatNebius, NebiusEmbeddings      # Nebius LLM
和嵌入模型
from langchain_community.graphs import Neo4jGraph              # Neo4j图数据
库接口
from langchain_community.vectorstores import FAISS             # FAISS向量存
储（内存版）
from langchain.docstore.document import Document               # 文档数据结构
from langchain_core.prompts import ChatPromptTemplate          # 结构化提示模
板

# ==================== LangGraph 组件导入 ====================
# LangGraph提供基于图的状态机框架，用于构建复杂的智能体工作流
from langgraph.graph import StateGraph, END                    # 状态图核心组
件
from typing_extensions import TypedDict                        # 类型字典定义

# ==================== 可视化与调试工具 ====================
# Rich库提供增强的终端输出格式，便于调试和演示
from rich.console import Console                               # 富文本控制台
```

```
输出
from rich.markdown import Markdown                                  # Markdown格
式渲染

# ==================== API密钥与环境配置 ====================
load_dotenv()  # 从.env文件加载环境变量

# 配置LangChain追踪系统，用于监控和调试记忆堆栈智能体
os.environ["LANGCHAIN_TRACING_V2"] = "true"                         # 启用V2版本
追踪
os.environ["LANGCHAIN_PROJECT"] = "Agentic Architecture - Memory Stack
(Nebius)"  # 项目标识

# 验证关键环境变量是否存在，记忆堆栈架构需要Neo4j数据库支持，缺失时给出警告
required_vars = [
    "NEBIUS_API_KEY",       # Nebius LLM服务密钥
    "LANGCHAIN_API_KEY",    # LangChain追踪服务密钥
    "NEO4J_URI",            # Neo4j数据库地址
    "NEO4J_USERNAME",       # Neo4j数据库用户名
    "NEO4J_PASSWORD"        # Neo4j数据库密码
]

for var in required_vars:
    if var not in os.environ:
        print(f"Warning: Environment variable {var} not set.")

print("Environment variables loaded and tracing is set up.")
```

阶段 1：构建记忆组件

这是架构的核心部分。**定义记忆的数据结构，并建立与数据库的连接**。此外，**创建负责处理对话并生成新记忆的“记忆生成器”智能体**。

```
# ==================== 控制台与LLM初始化 ====================
# 初始化Rich控制台实例，用于增强终端输出格式
console = Console()

# 使用Mixtral 8x22B模型，具有较强的指令理解和结构化输出能力
llm = ChatNebius(model="mistralai/Mixtral-8x22B-Instruct-v0.1",
temperature=0)

# 初始化Nebius嵌入模型，用于将文本转换为向量表示，嵌入向量将用于FAISS向量存储的相似性检
索
embeddings = NebiusEmbeddings()

# ==================== 1. 向量存储 - 情节记忆 ====================
# 情节记忆(Episodic Memory)：存储具体的事件和交互摘要
# 这种记忆类型用于检索历史对话上下文，理解用户偏好和历史行为
# 在真实应用中，应该持久化存储（如使用Redis、PostgreSQL等）
try:
```

```
    # 初始化FAISS向量存储，使用引导文档启动
    # FAISS (Facebook AI Similarity Search) 是高效的向量相似性搜索库
    episodic_vector_store = FAISS.from_texts(
        ["Initial document to bootstrap the store"],   # 引导文档，确保存储非
空
        embeddings                                     # 嵌入模型
    )
except ImportError:
    console.print("[bold red]FAISS not installed. Please run `pip install
faiss-cpu`.")
    episodic_vector_store = None

# ==================== 2. 图数据库 - 语义记忆 ====================
# 语义记忆(Semantic Memory): 存储实体、概念及其关系
# 使用Neo4j图数据库，支持复杂的关系查询和推理
# 这种记忆类型用于知识图谱构建、实体关系发现和因果推理

try:
    # 创建Neo4j图数据库连接
    graph = Neo4jGraph(
        url=os.environ.get("NEO4J_URI"),           # 数据库连接地址
        username=os.environ.get("NEO4J_USERNAME"), # 用户名
        password=os.environ.get("NEO4J_PASSWORD")  # 密码
    )
    # 清空图数据库，确保每次运行从干净状态开始
    graph.query("MATCH (n) DETACH DELETE n")
except Exception as e:
    console.print(f"[bold red]Failed to connect to Neo4j: {e}. Please check
your credentials and connection.")
    graph = None

# ==================== 3. 记忆提取数据模型 ====================
# 定义从对话中提取知识的Pydantic模型
# 这些模型定义了知识图谱的结构: 节点(Node)和关系(Relationship)

class Node(BaseModel):
    """
    知识图谱节点模型，节点代表实体（如人、公司、概念）或类别
    属性:id: 节点唯一标识符（如人名、公司代码、概念名）
        type: 节点类型分类（如'User', 'Company', 'InvestmentPhilosophy'）
        properties: 节点的属性字典（如年龄、市值、描述等）
    """
    id: str = Field(description="Unique identifier for the node, which can
be a person's name, a company ticker, or a concept.")
    type: str = Field(description="The type of the node (e.g., 'User',
'Company', 'InvestmentPhilosophy').")
    properties: Dict[str, Any] = Field(description="A dictionary of
properties for the node.")

class Relationship(BaseModel):
    """
    知识图谱关系模型，关系定义两个节点之间的连接，如"用户_投资_公司"
    属性:source: 关系源节点（起始节点）
        target: 关系目标节点（结束节点）
```

```
        type: 关系类型（如'IS_A', 'INTERESTED_IN', 'WORKS_AT'）
        properties: 关系属性（如权重、时间戳等）
    """
    source: Node = Field(description="The source node of the
relationship.")
    target: Node = Field(description="The target node of the
relationship.")
    type: str = Field(description="The type of the relationship (e.g.,
'IS_A', 'INTERESTED_IN').")
    properties: Dict[str, Any] = Field(description="A dictionary of
properties for the relationship.")

class KnowledgeGraph(BaseModel):
    """
    知识图谱提取结果模型，表示从单次对话中提取的结构化知识集合
    属性:relationships: 要添加到知识图谱的关系列表
    """
    relationships: List[Relationship] = Field(description="A list of
relationships to be added to the knowledge graph.")

# ==================== 4. 记忆创建智能体 ====================
# "记忆制造者" (Memory Maker) 负责从对话中提取和存储记忆
# 该智能体同时处理两种记忆类型:
# - 情节记忆: 对话摘要 (用于语义检索)
# - 语义记忆: 实体关系 (用于知识图谱)

def create_memories(user_input: str, assistant_output: str):
    """
    记忆创建函数 - 从对话中提取并存储多层次记忆
    两种记忆机制:
    1. 情节记忆(短时→长时): 将对话内容摘要化并存储到向量数据库
    2. 语义记忆(结构化知识): 从对话中提取实体关系并存储到图数据库
    双轨记忆机制实现了:
    - 快速检索: 通过向量相似性找到相关历史对话
    - 深度推理: 通过图查询发现实体间的隐含关系

    参数:user_input: 用户的输入消息
        assistant_output: 智能体的响应消息
    返回:None (记忆直接存储到全局向量存储和图数据库中)
    """

    # 组合完整的对话交互
    conversation = f"User: {user_input}\nAssistant: {assistant_output}"

    # ==================== 4a. 创建情节记忆 ====================
    # 情节记忆: 对话摘要, 用于后续检索
    console.print("--- Creating Episodic Memory (Summary) ---")

    # 构建摘要提示模板
    summary_prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a summarization expert. Create a concise, one-
sentence summary of the following user-assistant interaction. This summary
will be used as a memory for future recall."),
        ("human", "Interaction:\n{interaction}")
```

```
    ])

    # 创建摘要处理链
    summarizer = summary_prompt | llm

    # 生成对话摘要
    episodic_summary = summarizer.invoke({"interaction":
conversation}).content

    # 创建文档对象，包含摘要内容和唯一ID，使用uuid作为文档ID，便于后续去重和管理
    new_doc = Document(
        page_content=episodic_summary,
        metadata={"created_at": uuid.uuid4().hex}
    )

    # 将摘要添加到向量存储
    episodic_vector_store.add_documents([new_doc])
    console.print(f"[green]Episodic memory created:[/green]
'{episodic_summary}'")

    # ==================== 4b. 创建语义记忆 ====================
    # 语义记忆：结构化知识图谱，用于关系推理
    console.print("--- Creating Semantic Memory (Graph) ---")

    # 配置LLM为结构化输出模式，强制返回KnowledgeGraph格式
    extraction_llm = llm.with_structured_output(KnowledgeGraph)

    # 构建知识提取提示模板
    extraction_prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a knowledge extraction expert. Your task is to
identify key entities and their relationships from a conversation and model
them as a graph. Focus on user preferences, goals, and stated facts."),
        ("human", "Extract all relationships from this
interaction:\n{interaction}")
    ])

    # 创建知识提取处理链
    extractor = extraction_prompt | extraction_llm

    try:
        # 从对话中提取知识图谱
        kg_data = extractor.invoke({"interaction": conversation})
        # 将提取的关系添加到Neo4j图数据库
        if kg_data.relationships:
            for rel in kg_data.relationships:
                # add_graph_documents方法将关系转换为Neo4j的节点和边
                graph.add_graph_documents([rel], include_source=True)
            console.print(f"[green]Semantic memory created:[/green] Added
{len(kg_data.relationships)} relationships to the graph.")
        else:
            console.print("[yellow]No new semantic memories identified in
this interaction.[/yellow]")

    except Exception as e:
```

```
        console.print(f"[red]Could not extract or save semantic memory: {e}
[/red]")

# ==================== 5. 初始化验证 ====================
# 检查记忆组件是否成功初始化，只有两个记忆系统都可用时，才能正常工作
if episodic_vector_store and graph:
    print("Memory components initialized successfully.")
else:
    print("Warning: Some memory components failed to initialize.
Functionality may be limited.")
```

阶段 2：记忆增强型智能体

构建使用这套记忆系统的智能体。使用 LangGraph 来定义一个清晰、有状态的工作流程：**检索记忆、利用这些记忆生成响应，最后用最新的交互内容更新记忆。**

```
# ==================== 记忆增强智能体状态定义 ====================
# 定义记忆增强智能体的状态数据结构，该状态实现了记忆的检索-生成-更新闭环
class AgentState(TypedDict):
    """
    记忆增强智能体状态数据结构
    该状态设计体现了"检索增强生成"(RAG)的核心流程：
    1. user_input: 用户输入，作为记忆检索的查询
    2. retrieved_memories: 从记忆系统检索到的相关信息
    3. generation: 基于检索结果生成的最终响应
    """
    user_input: str                    # 用户当前输入
    retrieved_memories: Optional[str]  # 检索到的记忆（情节+语义）
    generation: str                    # 生成的响应

# ==================== 记忆检索节点 ====================
# 定义记忆检索节点，负责从两种记忆系统中获取相关信息
def retrieve_memory(state: AgentState) -> Dict[str, Any]:
    """
    记忆检索节点 - 从情节记忆和语义记忆中检索相关信息
    双通道记忆检索：
    1. 情节记忆检索：使用向量相似性搜索，找到相关的历史对话摘要
    2. 语义记忆检索：使用图数据库全文索引，查询相关的实体关系
    检索策略：
    - 情节记忆：使用用户输入作为查询，检索最相似的k条历史记录
    - 语义记忆：将用户输入分词，对每个关键词进行全文搜索

    参数:state: 当前智能体状态，包含用户输入
    返回:dict: 包含检索到的记忆内容的状态更新
    """
    console.print("--- Retrieving Memories ---")
    user_input = state['user_input']

    # ==================== 从情节记忆检索 ====================
    # 使用向量相似性搜索，找到与当前输入最相关的历史对话摘要
```

```
    retrieved_docs = episodic_vector_store.similarity_search(user_input,
k=2)
    episodic_memories = "\n".join([doc.page_content for doc in
retrieved_docs])

    # ==================== 从语义记忆检索 ====================
    # 从知识图谱中检索相关的实体关系
    # 使用全文索引进行关键词搜索，返回相关的节点和关系
    try:
        graph_schema = graph.get_schema
        # 使用全文索引进行关键词检索
        # UNWIND将关键词列表展开，为每个关键词执行全文搜索
        # db.index.fulltext.queryNodes是Neo4j的全文索引查询函数
        # MATCH查询返回相关节点及其连接的关系，限制5条结果
        semantic_memories = str(graph.query("""
            UNWIND $keywords AS keyword
            CALL db.index.fulltext.queryNodes("entity", keyword) YIELD
node, score
            MATCH (node)-[r]-(related_node)
            RETURN node, r, related_node LIMIT 5
            """, {'keywords': user_input.split()}))
    except Exception as e:
        semantic_memories = f"Could not query graph: {e}"

    # 组合两种记忆的检索结果
    retrieved_content = f"Relevant Past Conversations (Episodic
Memory):\n{episodic_memories}\n\nRelevant Facts (Semantic
Memory):\n{semantic_memories}"
    console.print(f"[cyan]Retrieved Context:\n{retrieved_content}[/cyan]")

    return {"retrieved_memories": retrieved_content}

# ==================== 响应生成节点 ====================
# 定义响应生成节点，基于检索到的记忆生成个性化回答
def generate_response(state: AgentState) -> Dict[str, Any]:
    """
    响应生成节点 - 使用检索到的记忆生成个性化响应
    实现了"检索增强生成"(RAG)的核心：
    1. 将检索到的记忆作为上下文
    2. 提示LLM利用记忆生成个性化回答
    3. 特别强调尊重用户的偏好（从语义记忆中提取）

    参数:state: 当前智能体状态，包含用户输入和检索到的记忆
    返回:dict: 包含生成响应的状态更新
    """
    console.print("--- Generating Response ---")

    # 构建生成提示模板
    # 系统提示：强调个性化响应和尊重用户偏好
    # 用户提示：包含用户问题和检索到的记忆
    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a helpful and personalized financial assistant.
Use the retrieved memories to inform your response and tailor it to the
user. If the memories indicate a user's preference (e.g., they are a
```

```
conservative investor), you MUST respect it."),
        ("human", "My question is: {user_input}\n\nHere are some memories
that might be relevant:\n{retrieved_memories}")
    ])

    # 创建生成处理链
    generator = prompt | llm

    # 生成响应
    generation = generator.invoke(state).content
    console.print(f"[green]Generated Response:\n{generation}[/green]")

    return {"generation": generation}

# ==================== 记忆更新节点 ====================
# 定义记忆更新节点，将本次交互存储到记忆系统
def update_memory(state: AgentState) -> Dict[str, Any]:
    """
    记忆更新节点 - 将当前交互存储到记忆系统
    该节点实现记忆的持续学习：
    1. 获取用户输入和智能体生成的响应
    2. 调用create_memories函数提取和存储新知识
    3. 更新两种记忆系统（情节记忆和语义记忆）

    参数:state: 当前智能体状态，包含用户输入和生成的响应
    返回:dict: 空状态更新（记忆更新不影响当前对话的其他状态）
    """
    console.print("--- Updating Memory ---")

    # 调用记忆创建函数，提取和存储新知识
    # - 生成对话摘要，存储到情节记忆（FAISS）
    # - 提取实体关系，存储到语义记忆（Neo4j图数据库）
    create_memories(state['user_input'], state['generation'])

    return {}

# ==================== 记忆增强智能体图构建 ====================
# 构建记忆增强智能体的计算图，该图实现了"检索-生成-更新"的完整闭环

# 初始化状态图构建器
workflow = StateGraph(AgentState)

# 添加三个核心节点
workflow.add_node("retrieve", retrieve_memory)   # 记忆检索节点
workflow.add_node("generate", generate_response) # 响应生成节点
workflow.add_node("update", update_memory)       # 记忆更新节点

# 定义执行流程：检索 → 生成 → 更新 → 结束
# 这是典型的RAG工作流，确保每次交互都：
# 1. 利用历史知识（检索）
# 2. 生成响应（生成）
# 3. 学习新知识（更新）
workflow.set_entry_point("retrieve")      # 入口点：记忆检索
workflow.add_edge("retrieve", "generate") # 检索完成后生成响应
```

```
workflow.add_edge("generate", "update")   # 生成完成后更新记忆
workflow.add_edge("update", END)          # 更新完成后结束

# ==================== 图编译与应用初始化 ====================
# 将构建的图结构编译为可执行的应用实例
memory_agent = workflow.compile()

# 输出编译成功状态
print("Memory-augmented agent graph compiled successfully.")
```

阶段 3：演示与检查

观察智能体的实际运行效果。模拟一个**多轮对话**：前两轮对话将用于初始化记忆，第三轮对话将测试智能体是否能利用这些记忆生成个性化响应。最后，直接查看数据库，检查所生成的记忆内容。

```
# ==================== 交互执行函数定义 ====================
def run_interaction(query: str) -> str:
    """
    执行单次用户交互的函数
    该函数封装了与记忆增强智能体的交互过程：接收用户输入 -> 调用记忆增强智能体处理 ->
返回生成的响应
    这是记忆堆栈架构的客户端接口，每次调用都会触发完整的检索-生成-更新闭环。

    参数:query: 用户输入的查询文本
    返回:str: 智能体生成的响应内容
    """
    # 调用记忆增强智能体
    # 智能体会自动执行以下流程：
    # 1. 检索记忆（情节记忆 + 语义记忆）
    # 2. 基于检索结果生成响应
    # 3. 更新记忆（存储本次交互）
    result = memory_agent.invoke({"user_input": query})
    return result['generation']

# ==================== 交互1：记忆播种 ====================
# 第一次交互：向智能体介绍用户的基本信息
# 这些信息将被存储到记忆系统中：
# - 情节记忆：存储对话摘要
# - 语义记忆：存储实体关系（用户偏好、投资风格等）
console.print("\n--- 💬 INTERACTION 1: Seeding Memory ---")
run_interaction("Hi, my name is Alex. I'm a conservative investor, and I'm
mainly interested in established tech companies.")

# ==================== 交互2：特定问题查询 ====================
# 第二次交互：询问对特定股票的看法，此时智能体已经拥有用户偏好的记忆
# 预期行为：
# 1. 检索记忆：找到用户是保守型投资者的信息
# 2. 结合记忆：在分析Apple时考虑保守型投资偏好
# 3. 生成响应：提供符合保守型投资者需求的Apple分析
#    可能强调Apple的稳定性、股息、成熟业务等，而非高增长、高风险的分析
```

```
console.print("\n--- 💬 INTERACTION 2: Asking a specific question ---")
run_interaction("What do you think about Apple (AAPL)?")

# ==================== 交互3：记忆测试 ====================
# 第三次交互：测试智能体是否真正记住了用户偏好，这是一个隐含记忆测试，不直接提及之前的对话
# 如果智能体工作正常，应该：
# - 识别用户是保守型投资者
# - 推荐同样是成熟、稳定的科技公司，例如：Microsoft (MSFT), Google (GOOGL), Intel
(INTC) 等
# - 避免推荐高风险、高增长的初创公司
# 这验证了记忆系统的核心能力：跨对话的上下文理解+用户偏好的持续记忆+个性化推荐的连贯性
console.print("\n--- 🧠 INTERACTION 3: THE MEMORY TEST ---")
run_interaction("Based on my goals, what's a good alternative to that
stock?")
```

检查记忆存储

深入底层，**直接查询数据库，查看智能体所创建的记忆。**

```
# ==================== 情节记忆检查 ====================
# 检查向量存储中存储的情节记忆（对话摘要），验证智能体是否正确存储了历史交互的摘要信息
console.print("--- 🔍 Inspecting Episodic Memory (Vector Store) ---")

# 执行相似性搜索，检索与"用户投资策略"相关的历史记忆
retrieved_docs = episodic_vector_store.similarity_search("User's investment
strategy", k=3)

# 输出检索到的情节记忆
for i, doc in enumerate(retrieved_docs):
    print(f"{i+1}. {doc.page_content}")

# ==================== 语义记忆检查 ====================
# 检查图数据库中存储的语义记忆（实体关系知识图谱），验证智能体是否正确提取和存储了用户偏
好、目标等结构化知识
console.print("\n--- 🕸 Inspecting Semantic Memory (Graph Database) ---")

# 输出图数据库的完整架构
# get_schema返回Neo4j数据库中的所有节点标签、关系类型和属性定义，这有助于理解知识图谱
的数据模型
print(f"Graph Schema:\n{graph.get_schema}")

# 执行Cypher查询，检查用户节点及其关系
# 查询解析：
# MATCH (n:User)-[r:INTERESTED_IN|HAS_GOAL]->(m)
# - 查找所有类型为User的节点n
# - 查找从User出发的关系r，类型为INTERESTED_IN或HAS_GOAL
# - 查找关系指向的目标节点m
# RETURN n, r, m - 返回节点、关系和目标节点
query_result = graph.query("MATCH (n:User)-[r:INTERESTED_IN|HAS_GOAL]->(m)
RETURN n, r, m")
print(f"Relationships in Graph:\n{query_result}")
```

结论

在本节笔记本中，我们成功构建了一个具备复杂长期记忆系统的智能体。演示结果清晰地展示了这一架构的强大之处：

- **无状态模式的失败：** 当被问及“根据我的目标，有什么好的替代选择？”时，标准智能体会失败，因为它对用户的目标没有任何记忆。
- **记忆增强模式的成功：** 我们的智能体之所以成功，是因为它能够：
  1. **情景记忆检索：** 它**检索**到了第一次对话的摘要：“用户 Alex 介绍自己是一名保守型投资者……”
  2. **语义记忆检索：** 它查询**图数据库**并找到了**结构化事实**：`（用户： Alex） -[拥有目标]-> （投资理念： 保守型）`。
  3. **综合生成：** 它利用这些**整合的上下文信息**，提供了**高度相关且个性化**的建议（微软），并明确引用了用户的保守型目标。

这种结合回忆 **“发生了什么”（情景记忆）与“已知什么”（语义记忆）** 的方式，是一种强大的范式，使我们能够超越简单的交互式智能体，创造出真正的、能够不断学习的伙伴。尽管在规模化管理这类记忆时面临诸如修剪和整合等挑战，但我们在此构建的基础架构，已然朝着更智能、更个性化的 AI 系统迈出了重要一步。